



**TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ ĐÔNG Á**
EAST ASIA UNIVERSITY OF TECHNOLOGY

International Training & Cooperation Institute

Introduction to Blockchain Technology

Lesson 11 – Contract Events & Dapps Frontend Interaction



Contract Events & Dapps Frontend Interaction



- 1. Events in Ethereum**
- 2. DApp Architecture and Interaction Flow**
- 3. Web3 Libraries**
- 4. Contract and Frontend Integration**
- 5. Exercises**

1. Events in Ethereum

What is an Event?

- ❖ A log recorded on blockchain
- ❖ Emitted when a contract executes a function
- ❖ Used by off-chain systems (frontend, backend)

Events are stored in transaction logs, not contract storage.

Example:

Event emitted when ETH is sent

No data stored in state

Visible after transaction is mined

```
event Deposited(address sender, uint amount);  
  
function deposit() public payable {  
    emit Deposited(msg.sender, msg.value);  
}
```

1. Events in Ethereum

Event vs State Variable

Events describe what happened, not what is stored now.

Feature	State Variable	Event
Stored in blockchain state	Yes	No
Stored in transaction logs	No	Yes
Gas cost	High	Lower
Query after deployment	Yes	No (only history)
Used by frontend	Sometimes	Very often

- ❖ Remix IDE (Logs panel)
- ❖ Blockchain explorers (Etherscan)
- ❖ dApps
- ❖ Node providers (RPC / WebSocket)

[illegible]

2. DApp Architecture and Interaction Flow



Common DApp Flow:

Frontend → Wallet (MetaMask) → Web3
Library → RPC Node (Infura/Alchemy/Local)
→ Smart Contract

❖ Events travel in the opposite direction

Smart Contract

↓ emit event

Transaction

↓ logs

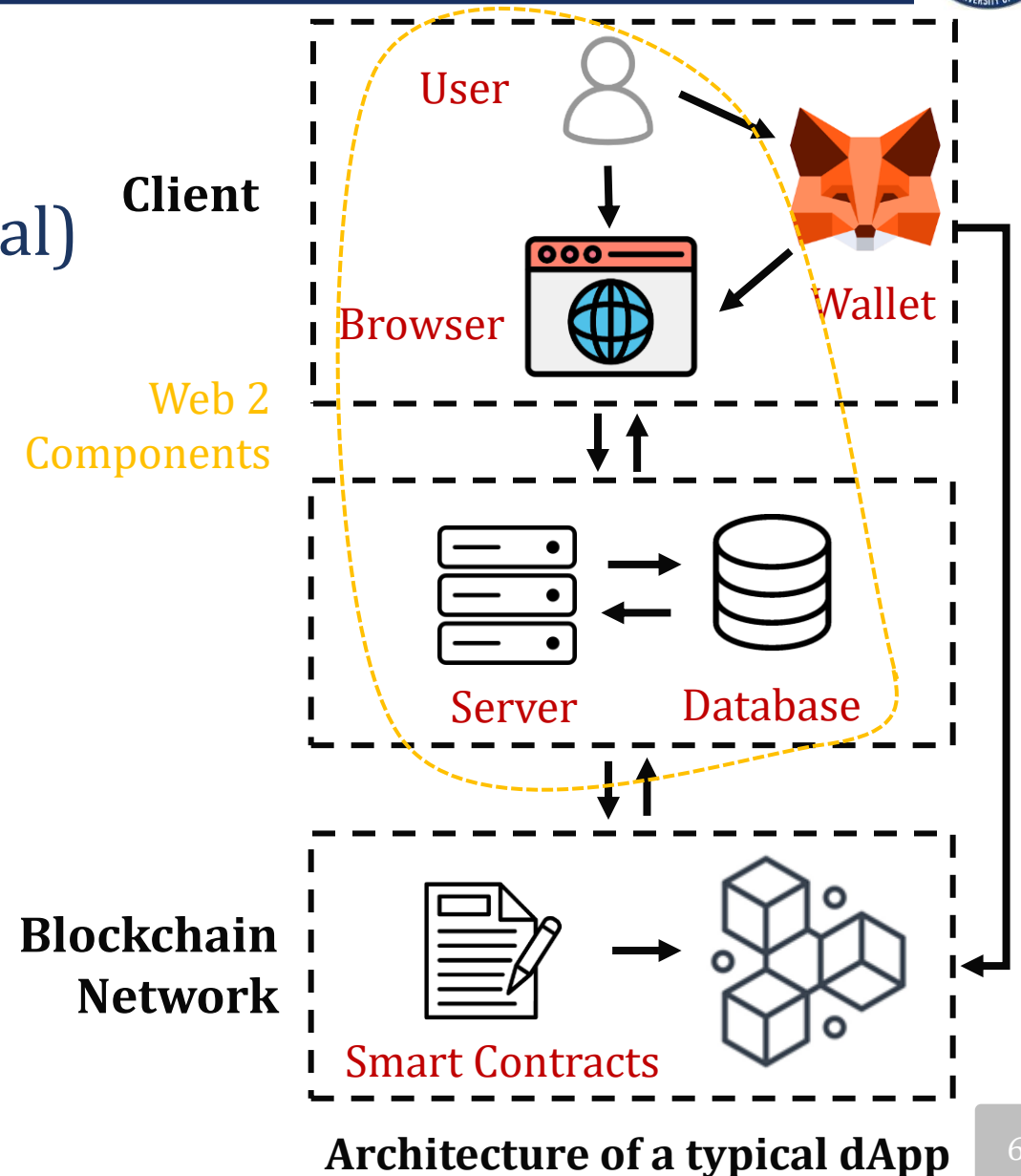
Ethereum Node

↓ RPC/WebSocket

Web3 Library

↓ callback

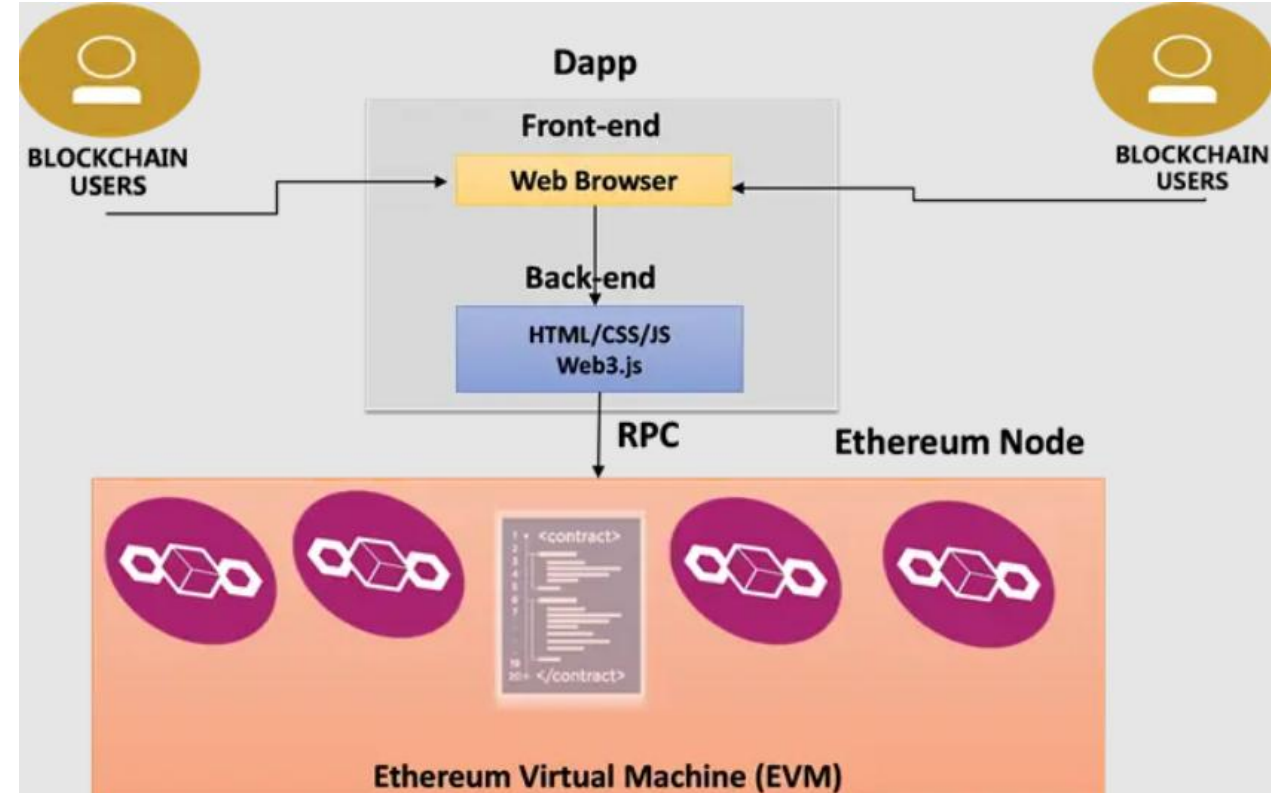
Frontend UI



2. DApp Architecture and Interaction Flow

DApp frontend:

- ❖ A normal web page
- ❖ Connected to a blockchain wallet
- ❖ Used to call smart contract functions
- ❖ Runs off-chain
- ❖ Can be modified without redeploying smart contracts
- ❖ Communicates with blockchain via wallet and RPC



2. DApp Architecture and Interaction Flow

How Frontends Talk to Ethereum

Frontend does NOT talk directly to the blockchain.

It talks through:

- ❖ A wallet (MetaMask)
- ❖ A Web3 library

The library handles:

- ❖ ABI encoding
- ❖ **JSON-RPC** (**R**emote **P**rocedure **C**all node is a server that acts as a bridge between applications and a blockchain network)
- ❖ Transaction signing
- ❖ Event log decoding

3. Web3 Libraries

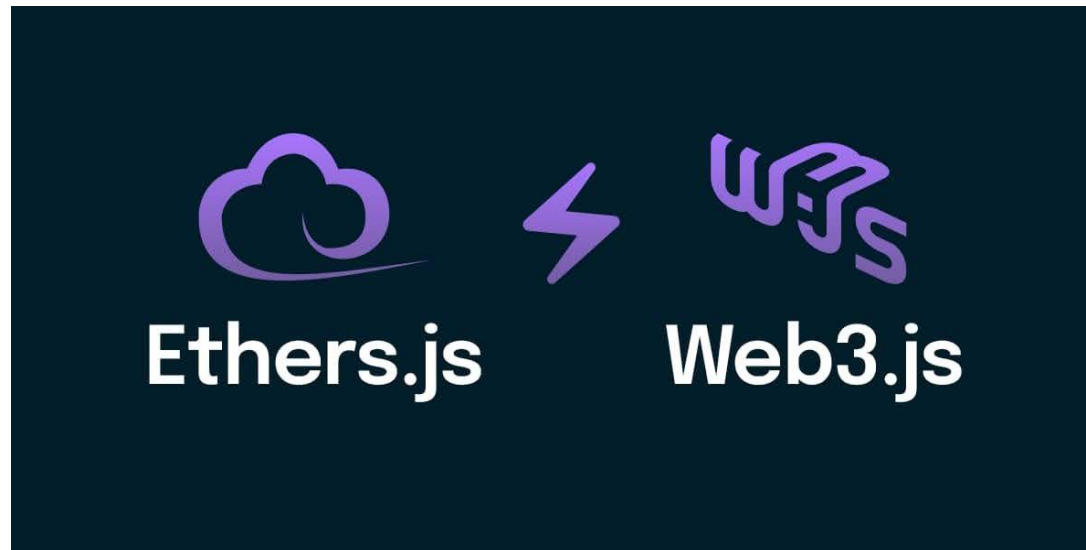
What does a Web3 library do?

- ❖ Encode Solidity function calls (ABI)
- ❖ Send JSON-RPC requests
- ❖ Sign transactions via wallet
- ❖ Decode return values and events

3. Web3 Libraries



	Ethers.js	Web3.js
API	Modern async/await	Callback + Promise
Size	Lightweight	Heavy



In this course, we use **Ethers.js**: <https://www.npmjs.com/package/ethers>

3. Web3 Libraries

Using **Ethers.js**, the frontend can:

- ❖ Connect to MetaMask
- ❖ Get the user wallet address
- ❖ Send transactions
- ❖ Listen to contract events

4. Contract and Frontend Integration



Contract code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract MessageApp {
    string public message;
    event MessageUpdated(address user, string newMessage);

    constructor(string memory _msg) {
        message = _msg;
    }

    function updateMessage(string memory _msg) public {
        message = _msg;
        emit MessageUpdated(msg.sender, _msg);
    }
}
```

4. Contract and Frontend Integration

Frontend Code: Must be run through a web server instead of opening the file directly.

VS Code Live Server

1. If you're using Visual Studio Code:
2. Install the Live Server extension.
3. Right-click your index.html file.
4. Choose "Open with Live Server".
5. It will launch automatically at <http://127.0.0.1:5500>

4. Contract and Frontend Integration

Frontend Code: Import Web3 Library, define contract address and ABI

P1

```
<html>
  <body>
    <h3>Message DApp</h3>
    <button id="connect">Connect Wallet</button>
    <p>Message: <span id="msg"></span></p>
    <input id="inputMsg" placeholder="New message">
    <button id="update">Update</button>
    <div id="log"></div>
    <script type="module">

      import { ethers } from "https://cdn.jsdelivr.net/npm/ethers@6.10.0/dist/ethers.min.js";

      const address = "/* Paste Deployed Contract Address here */";
      const abi = [/* Paste Full Contract ABI here*/];
      function main() {
        if (!window.ethereum) {
          alert("Please install MetaMask!");
          return; }
      }

      main();
```

4. Contract and Frontend Integration

Frontend Code: Contract Connection & Calling Contract Function

P2

```
let contract;
async function connect() {
  const provider = new ethers.BrowserProvider(window.ethereum);
  await provider.send("eth_requestAccounts", []);
  const signer = await provider.getSigner();
  contract = new ethers.Contract(address, abi, signer);
  await loadMessage(); }

document.getElementById("connect").onclick = connect;
document.getElementById("update").onclick = async () => {
  const text = document.getElementById("inputMsg").value;
  const tx = await contract.updateMessage(text);
  await tx.wait();
  await loadMessage(); }
async function loadMessage() {
  if (!contract) return;
  try {
    const m = await contract.message();
    document.getElementById("msg").innerText = m;
  } catch (error) { console.error("Failed to load message:", error); }}
</script>
</body>
</html>
```

4. Contract and Frontend Integration

Frontend Code: Listen to Blockchain Event

- ❖ *contract.on(eventName, callback)*: listens for new events in real-time via WebSocket/RPC subscriptions, for **live** data
- ❖ *contract.queryFilter(filter, fromBlock, toBlock)*: scans transaction logs backward from a block range, for **historical** data

P2.1

```
async function connect() {  
  /* Existing code */  
  contract.on("MessageUpdated", (sender, newMessage) => {  
    document.getElementById("log").innerHTML +=  
      `

${sender}: ${newMessage}</p>`;   
  });  
}


```

5. Exercises

Ex 1: Build a User Registration smart contract and simple web interface

Create a smart contract that allows anyone to register with **userName**

- ❖ Store the username of the user who registers last
- ❖ Build a frontend to interact with smart contract functions

Ex 2: Upgrade the contract to store multiple users

Modify **Ex 1** contract so that:

- ❖ Each address can register once
- ❖ Store username for each address
- ❖ Emit **UserRegistered** event on every registration

5. Exercises



Ex 3: Build a webpage that listens to **UserRegistered** events and displays a live list of registered users.

From **Ex 2**, build a webpage to listen to all **UserRegistered** events, then display the list of all registered student based on events listened.